

Abshaar Chronological ML Roadmap

A step-by-step curriculum for building a South Asian poetry translation and explanation platform

Prepared for Rauf Nawaz
Project: Abshaar
Date: June 7, 2026

How to use this PDF

Read this guide in order. Each phase tells you what to learn, what to build inside Abshaar, what resources to use, and what must be true before you move on. The checklists are intentionally practical. If you can honestly check the boxes, you are ready for the next phase.

0. The destination

Abshaar is not just a translation app. It is a public-access literary technology project for South Asian poetry. The first serious version should let a reader open a poet such as Bulleh Shah, read the original text, see transliteration, compare literal and literary English translations, study cultural and Sufi references, and ask source-grounded questions.

The long-term system has eight layers:

Layer	What it does	Example in Abshaar
Corpus layer	Stores original poems, metadata, variants, and sources	Bulleh Shah poem in Shahmukhi, Gurmukhi, Roman, and English
Transliteration layer	Converts between scripts or displays multiple scripts	Shahmukhi to Roman transliteration
Translation layer	Produces literal and literary translations	Punjabi or Urdu to English
Interpretation layer	Explains metaphors, idioms, poetic form, Sufi concepts, and historical context	Ranjha as beloved, self, and possible sign of divine absorption
RAG layer	Retrieves approved notes before the model answers	Glossary entry plus poet note plus line annotation
Human-review layer	Lets people correct translations and explanations	Scholar or native speaker edits model draft
Fine-tuning layer	Teaches a model your preferred Abshaar output style	LoRA fine-tune on reviewed poem examples
Website layer	Makes the project public and usable	Bulleh Shah Explorer

The most important idea is this:

The model is not the whole project. The dataset, annotations, source-grounding, review process, and public interface are just as important as the model.

The chronological path

Phase	Main focus	Main Abshaar output
Phase 1	Setup, computing basics, GitHub	You can run code and update the repo
Phase 2	Python fundamentals	You can read and write JSON, CSV, and scripts
Phase 3	Corpus and dataset design	First structured poem entries and source registry
Phase 4	Data cleaning, Unicode, and validation	Scripts that check your data quality
Phase 5	ML foundations	You understand loss, gradient descent, overfitting, and evaluation
Phase 6	PyTorch and deep learning	You can read a training loop and train a toy model
Phase 7	NLP and transformers	You understand tokenization, embeddings, attention, and model types
Phase 8	Translation baselines	You compare IndicTrans2, NLLB, and LLM outputs
Phase 9	Evaluation and human review	You create a translation rubric and reviewed dataset
Phase 10	RAG for explanation	You build source-grounded question answering
Phase 11	Fine-tuning	You run a small LoRA or QLoRA experiment
Phase 12	Website MVP	You publish a small Bulleh Shah Explorer
Phase 13	Launch, governance, and growth	You make the project sustainable and trustworthy

Phase 1. Setup, computing basics, and project orientation

Recommended time: 1 to 2 weeks.

This phase is about becoming comfortable enough with your machine that code no longer feels like a black box. Do not start with AI. Start with the basic tools that every AI project uses.

Lecture 1.1 - What code, files, and folders are doing

What to learn

You need to understand the difference between:

- A file and a folder.
- A local file and a file on GitHub.
- A script and a notebook.
- A terminal command and a Python command.
- A text file, CSV file, JSON file, and Markdown file.

Why Abshaar needs this

Your project will eventually contain hundreds or thousands of small structured files. A poem may be stored in JSON. A glossary may be stored in JSONL. A source registry may be a CSV. A script may validate whether each poem has required metadata. If you do not understand files and folders, the project will become chaotic quickly.

Resources

- CS50's Introduction to Programming with Python: <https://cs50.harvard.edu/python/>
- GitHub Skills: <https://skills.github.com/>
- VS Code documentation: <https://code.visualstudio.com/docs>

Practice task

Create this local folder structure:

```
abshaar-practice/  
  README.md  
  data/  
    poems/  
    glossary/  
    sources/  
  scripts/  
  notebooks/
```

Write one sentence in `README.md` explaining what Abshaar is.

Checklist before moving on

- ☐ I can create folders and files without help.
- ☐ I can open a folder in VS Code.
- ☐ I can tell the difference between a `.py`, `.ipynb`, `.json`, `.csv`, and `.md` file.
- ☐ I can run a simple terminal command such as `pwd`, `ls`, `cd`, and `mkdir`.
- ☐ I have created a local practice folder for Abshaar.

Lecture 1.2 - Git and GitHub basics

What to learn

Git is version control. GitHub is where your project can live publicly or privately. You need to know:

- `git clone`
- `git status`
- `git add`
- `git commit`
- `git push`
- `git pull`
- branches
- pull requests
- README files
- issues

Why Abshaar needs this

Abshaar should be open-source and collaborative. If a friend, scholar, or developer helps later, GitHub becomes the shared memory of the project. It also lets you track why a translation, schema, or model experiment changed.

Resources

- GitHub Skills: <https://skills.github.com/>

- Pro Git book: <https://git-scm.com/book/en/v2>
- Abshaar repository: <https://github.com/RaufNawaz/Abshaar>

Practice task

Clone the Abshaar repository. Create a file called `notes/learning-log.md` and write what you learned in Phase 1. Commit and push the change.

```
git clone https://github.com/RaufNawaz/Abshaar.git
cd Abshaar
git status
mkdir -p notes
printf "# Learning Log\n\nPhase 1 started.\n" > notes/learning-log.md
git add notes/learning-log.md
git commit -m "Add learning log"
git push
```

Checklist before moving on

- ☐ I have cloned the Abshaar repo.
- ☐ I know how to check which files changed.
- ☐ I have made at least one commit.
- ☐ I have pushed at least one commit to GitHub.
- ☐ I can explain why GitHub is useful for a public poetry and ML project.

Project checkpoint for Phase 1

By the end of Phase 1, you should have:

- ☐ A working coding environment.
- ☐ VS Code installed and usable.
- ☐ Python installed.
- ☐ Git installed.
- ☐ Access to the Abshaar repo.
- ☐ A first commit in the repo or in a practice repo.

Phase 2. Python fundamentals for a humanities student

Recommended time: 4 to 6 weeks.

The goal is not to become a software engineer immediately. The goal is to become independent enough to read, run, modify, and debug scripts used in Abshaar.

Lecture 2.1 - Python syntax and basic thinking

What to learn

- variables
- strings
- integers and floats
- lists
- dictionaries
- conditionals
- loops
- functions
- print statements
- comments

Why Abshaar needs this

Every later step depends on Python. You will use Python to load poems, normalize text, call translation models, create datasets, run evaluations, and eventually fine-tune models.

Resources

- CS50 Python: <https://cs50.harvard.edu/python/>
- Kaggle Python: <https://www.kaggle.com/learn/python>
- Python official tutorial: <https://docs.python.org/3/tutorial/>

Practice task

Create a Python dictionary for one poem:

```
poem = {  
    "title": "Ranjha Ranjha Kardi",  
    "poet": "Bulleh Shah",  
    "language": "Punjabi",  
    "script": "Shahmukhi",  
    "status": "draft"  
}  
  
print(poem["title"])  
print(poem["poet"])
```

Checklist before moving on

- ☐ I can write and run a `.py` file.
- ☐ I understand strings, lists, and dictionaries.
- ☐ I can write a function.
- ☐ I can use a `for` loop.
- ☐ I can read an error message without panicking.
- ☐ I can represent a poem as a Python dictionary.

Lecture 2.2 - Files, JSON, CSV, and UTF-8

What to learn

- reading and writing text files
- JSON
- JSONL
- CSV
- UTF-8 encoding
- why non-English text needs careful encoding

Why Abshaar needs this

Punjabi, Urdu, Saraiki, Persian, and Arabic-script material will break if you treat text carelessly. You must always save text as UTF-8 and check that scripts display correctly.

Example original line

رانجھا رانجھا کردی نی میں آپے رانجھا ہوئی

Roman transliteration:


```
Ranjha Ranjha kardi ni, main aape Ranjha hoi
```

Practice task

Write a poem entry to JSON while preserving Arabic-script text:

```
import json

poem = {
    "id": "bulleh_001",
    "title": "Ranjha Ranjha Kardi",
    "poet": "Bulleh Shah",
    "language": "Punjabi",
    "script": "Shahmukhi",
    "original_text": "رانجھا رانجھا کردی نی میں آپے رانجھا ہوئی",
    "roman_transliteration": "Ranjha Ranjha kardi ni, main aape Ranjha hoi",
    "review_status": "draft"
}

with open("bulleh_001.json", "w", encoding="utf-8") as f:
    json.dump(poem, f, ensure_ascii=False, indent=2)
```

Checklist before moving on

- ☐ I can save non-English text without it turning into question marks.
- ☐ I can write a JSON file from Python.
- ☐ I can load a JSON file into Python.
- ☐ I know why `encoding="utf-8"` matters.
- ☐ I know why `ensure_ascii=False` matters for Urdu and Punjabi script.
- ☐ I have created one Abshaar poem JSON file.

Lecture 2.3 - Pandas and spreadsheets

What to learn

- loading CSV files
- viewing rows and columns
- filtering data
- saving CSV and JSONL
- basic data cleaning

Why Abshaar needs this

At the beginning, you will probably use spreadsheets to review translations. Later, those spreadsheets should become machine-readable datasets.

Resources

- Kaggle Pandas: <https://www.kaggle.com/learn/pandas>
- Pandas getting started: https://pandas.pydata.org/docs/getting_started/index.html

Practice task

Create a CSV with columns:

```
line_id,poet,title,original_text,model_translation,human_translation,notes,review_status
```

Then load it with Pandas:

```
import pandas as pd

df = pd.read_csv("translation_review.csv")
print(df.head())
print(df["review_status"].value_counts())
```

Checklist before moving on

- ☐ I can open a CSV in Python.
- ☐ I can filter rows in Pandas.
- ☐ I can save a cleaned CSV.
- ☐ I can explain when a spreadsheet is useful and when JSON is better.
- ☐ I have created an initial translation review spreadsheet.

Project checkpoint for Phase 2

By the end of Phase 2, your Abshaar folder should contain:

```
data/
  poems/
    bulleh_001.json
  reviews/
    translation_review.csv
scripts/
  load_poem.py
  csv_to_json.py
```

You should be able to:

- ☐ Load a poem JSON file.
- ☐ Print the original text and transliteration.
- ☐ Convert a simple CSV to JSON.
- ☐ Commit the files to GitHub.

Phase 3. Corpus design and dataset architecture

Recommended time: 4 to 6 weeks.

This phase is where Abshaar becomes a real project. A bad dataset will ruin even a strong model. A good dataset can make a modest model useful.

Lecture 3.1 - What a corpus is

What to learn

A corpus is a structured collection of texts. For Abshaar, a corpus is not only poem text. It includes:

- original text
- poet
- title
- language
- script
- genre
- source
- license status
- transliteration
- literal translation
- literary translation
- annotations
- glossary terms
- review status

Why Abshaar needs this

A translation model can only learn from what you give it. If your corpus does not record sources, scripts, line IDs, and review status, you will later be unable to evaluate or fine-tune responsibly.

Resources

- Hugging Face Datasets documentation: <https://huggingface.co/docs/datasets/index>
- Dataset card guide: <https://huggingface.co/docs/hub/datasets-cards>
- Data Version Control: <https://dvc.org/doc>

Practice task

Define a poem entry schema in plain English before writing code. For example:

```
Every poem must have:  
- id  
- title  
- poet  
- language  
- script  
- original_text  
- source_id  
- license_status  
- review_status
```

Checklist before moving on

- ☐ I can explain the difference between raw text and a corpus.
- ☐ I know why each poem needs a stable ID.
- ☐ I know why each line may need its own ID.
- ☐ I can explain why source and license fields are required.
- ☐ I have drafted the required fields for Abshaar poem entries.

Lecture 3.2 - The Abshaar poem schema

Recommended schema

```
{  
  "id": "bulleh_001",  
  "poet": "Bulleh Shah",  
  "poet_dates": "1680-1758",  
  "title": "Ranjha Ranjha Kardi",  
  "genre": "Kafi",  
  "language": "Punjabi",  
  "script": "Shahmukhi",  
  "original_text": "...",  
  "normalized_text": "...",  
  "roman_transliteration": "...",  
  "literal_translation": "...",  
  "literary_translation": "...",  
  "tashreeh": "...",  
  "glossary_terms": ["Ranjha", "Heer", "Ishq"],  
  "source_ids": ["source_001"],  
  "license_status": "needs_verification",  
  "review_status": "draft",  
  "reviewer_notes": ""  
}
```

Why this structure matters

You should keep separate fields for literal translation, literary translation, and explanation. If you mix them together, the model may later learn to over-interpret instead of translating carefully.

Checklist before moving on

- ☐ I have created a JSON schema for poems.
- ☐ I separate original text from normalized text.
- ☐ I separate literal translation from literary translation.
- ☐ I separate translation from *tashreeh*.
- ☐ I record source IDs rather than only pasting URLs into every poem.

Lecture 3.3 - Source registry and licensing

What to learn

You must track where every text comes from and what you are allowed to do with it.

Use categories like:

License status	Meaning	Allowed use
public_domain_confirmed	Text is in the public domain and source is clear	Can display and use in dataset
permission_granted	Rights holder gave permission	Use according to permission terms
open_license	License allows reuse, possibly with conditions	Follow license terms
unclear	Online but rights unknown	Do not train or republish yet
copyrighted_restricted	Rights reserved or modern translation	Do not scrape or train without permission

Important warning

Online availability is not the same as permission. For example, a website may show poetry or translations publicly, but that does not mean you can scrape, republish, or use it for training.

Possible reference sources to investigate

- Punjabi Kavita Bulleh Shah page: <https://www.punjabi-kavita.com/Baba-Bulleh-Bulleh-Shah.php>
- Sufinama Bulleh Shah page: <https://sufinama.org/poets/bulleh-shah/all>
- Rekhta Bulleh Shah page: <https://www.rekhta.org/poets/bulleh-shah/all?lang=ur>

- Archive.org collections: <https://archive.org/>

Treat these as sources to investigate, not automatic training data.

Source registry example

```
{
  "source_id": "source_001",
  "name": "Punjabi Kavita Bulleh Shah page",
  "url": "https://www.punjabi-kavita.com/Baba-Bulleh-Bulleh-Shah.php",
  "content_type": "original poetry text",
  "license_status": "needs_verification",
  "accessed_date": "2026-06-07",
  "notes": "Use for reference only until rights are checked."
}
```

Checklist before moving on

- ☐ I have a `source_registry.json` file.
- ☐ Every poem entry has at least one `source_id`.
- ☐ Every source has a license status.
- ☐ I understand that modern translations are usually copyrighted.
- ☐ I will not use copyrighted translations as training data without permission.

Lecture 3.4 - Line-level data

What to learn

Poetry should often be stored at both poem level and line level. Full poems are useful for reading. Lines and couplets are useful for translation experiments.

Line schema example

```
{
  "line_id": "bulleh_001_l001",
  "poem_id": "bulleh_001",
  "line_number": 1,
  "original_text": "رانجھا رانجھا کردی نی میں آپے رانجھا ہوئی",
  "roman_transliteration": "Ranjha Ranjha kardi ni, main aape Ranjha hoi",
  "literal_translation": "Saying Ranjha, Ranjha, I myself became Ranjha.",
  "literary_translation": "Calling Ranjha's name again and again, I became Ranjha myself.",
  "review_status": "draft"
}
```

Checklist before moving on

- ☐ I can explain why line-level IDs matter.

- ☐ I can split one poem into line entries.
- ☐ I can link every line back to a poem ID.
- ☐ I can store translation drafts separately from reviewed translations.
- ☐ I have created line-level entries for at least one poem.

Project checkpoint for Phase 3

By the end of Phase 3, you should have:

- ☐ 5 to 10 structured poem entries.
- ☐ A source registry.
- ☐ A line-level JSONL file.
- ☐ A first glossary file with 10 terms.
- ☐ A documented rule: no unlicensed modern translations in training data.

Phase 4. Data cleaning, Unicode, and validation

Recommended time: 3 to 5 weeks.

This phase teaches you to protect your dataset from silent errors. Silent errors are dangerous because the code may run but the data may be wrong.

Lecture 4.1 - Unicode and right-to-left text

What to learn

- Unicode
- UTF-8
- Arabic-script text
- diacritics
- invisible characters
- normalization
- whitespace cleanup
- right-to-left rendering

Why Abshaar needs this

Punjabi Shahmukhi, Urdu, Persian, and Arabic use Arabic-derived scripts. Text may include characters that look similar but are encoded differently. A model may treat two visually similar words as different token sequences.

Practical examples

You may need to normalize:

- Arabic Yeh versus Persian Yeh.
- Arabic Kaf versus Persian Keheh.
- extra spaces.
- line breaks.
- zero-width characters.
- optional diacritics.

Checklist before moving on

- [] I know what UTF-8 is.

- [] I know that two characters can look similar but be different in Unicode.
- [] I can detect empty strings and extra whitespace.
- [] I can preserve right-to-left text in JSON.
- [] I have created a small script to print the Unicode code points of a line.

Lecture 4.2 - Dataset validation scripts

What to learn

Validation means checking whether your data follows your rules.

Examples:

- every poem has an ID
- every poem has a source
- every source has a license status
- every line has original text
- no required field is blank
- no two poems share the same ID

Practice script

```
import json
from pathlib import Path

required_fields = [
    "id", "title", "poet", "language", "script",
    "original_text", "source_ids", "license_status", "review_status"
]

errors = []

for path in Path("data/poems").glob("*.json"):
    with open(path, encoding="utf-8") as f:
        poem = json.load(f)
        for field in required_fields:
            if field not in poem or poem[field] in ["", [], None]:
                errors.append(f"{path.name}: missing {field}")

if errors:
    print("Validation failed:")
    for error in errors:
        print("-", error)
else:
    print("All poem files passed validation.")
```

Checklist before moving on

- [] I can write a script that loops through files.

- ☐ I can check for missing fields.
- ☐ I can print useful error messages.
- ☐ I can run validation before committing data.
- ☐ I have a script named something like `validate_poems.py`.

Lecture 4.3 - Data quality levels

What to learn

Not all data should be treated equally. Use explicit quality labels.

Quality level	Meaning	Training use
raw	copied or collected but not checked	no
cleaned	formatted and source-tracked	maybe for display if allowed
auto_translated	generated by model	no as gold data
human_reviewed	corrected by competent reviewer	yes
expert_reviewed	reviewed by scholar or strong native speaker	best
published	approved for public site	yes with provenance

Checklist before moving on

- ☐ I know the difference between draft data and gold data.
- ☐ I do not treat model-generated translations as ground truth.
- ☐ I have added `review_status` to all poem or line entries.
- ☐ I have a way to mark `human_reviewed` examples.
- ☐ I understand why fine-tuning on low-quality data can make a model worse.

Project checkpoint for Phase 4

By the end of Phase 4, you should have:

- ☐ A validation script.
- ☐ A data quality labeling system.
- ☐ At least 10 poem entries that pass validation.
- ☐ At least 50 line entries that pass validation.
- ☐ A rule that auto-generated outputs are never mixed with reviewed gold data.

Phase 5. Machine learning foundations

Recommended time: 5 to 7 weeks.

This is where the phrase you heard about "linear regression" becomes useful. You do not need to manually write linear regression for Abshaar later, but linear regression teaches the core idea of model training.

Lecture 5.1 - The basic training loop

What to learn

A model learns by adjusting parameters to reduce error.

In simple form:

```
prediction = model(input)
loss = how wrong prediction is
training = adjust parameters to reduce loss
```

For linear regression:

```
prediction = weight * input + bias
loss = difference between prediction and correct answer
```

For translation:

```
input = original Punjabi or Urdu line
target = reviewed English translation
prediction = model-generated English tokens
loss = how unlikely the target tokens were under the model
```

Resources

- Google ML Crash Course: <https://developers.google.com/machine-learning/crash-course>
- Linear regression module: <https://developers.google.com/machine-learning/crash-course/linear-regression>
- Gradient descent module: <https://developers.google.com/machine-learning/crash-course/linear-regression/gradient-descent>
- Kaggle Intro to Machine Learning: <https://www.kaggle.com/learn/intro-to-machine-learning>

Checklist before moving on

- [] I can define model, parameter, prediction, target, and loss.
- [] I understand that training changes parameters.

- ☐ I understand that loss should usually go down during training.
- ☐ I can explain why linear regression is a teaching example, not the final Abshaar method.
- ☐ I can explain the Abshaar equivalent of input, target, prediction, and loss.

Lecture 5.2 - Gradient descent and learning rate

What to learn

Gradient descent is the method that moves parameters in the direction that reduces loss. The learning rate controls how large each update is.

Learning rate problem	What happens
Too high	training becomes unstable or loss explodes
Too low	training is very slow or barely learns
Reasonable	loss decreases steadily

Why Abshaar needs this

When you eventually fine-tune a model, you will set hyperparameters like learning rate, batch size, and epochs. You do not need to derive the math by hand, but you need to understand what these choices mean.

Checklist before moving on

- ☐ I can explain gradient descent in plain English.
- ☐ I know what learning rate controls.
- ☐ I know why too high a learning rate is dangerous.
- ☐ I know why too low a learning rate can waste time.
- ☐ I can read a loss curve and say whether training looks healthy.

Lecture 5.3 - Train, validation, test, and overfitting

What to learn

You never evaluate only on the data the model trained on.

Split	Purpose
Train	examples the model learns from
Validation	examples used to tune decisions during development
Test	held-out examples used for final evaluation

Overfitting means the model memorizes training examples but performs poorly on new examples.

Abshaar example

A model might become good at Bulleh Shah lines it has seen, but fail on a new Shah Hussain or Waris Shah poem. That is overfitting.

Checklist before moving on

- ☐ I can explain train, validation, and test splits.
- ☐ I can explain overfitting.
- ☐ I can explain why a model should be tested on unseen poems.
- ☐ I know why validation loss matters.
- ☐ I can split a CSV into train, validation, and test files.

Lecture 5.4 - Baselines and error analysis

What to learn

A baseline is the simplest system you compare against. For Abshaar, baselines might be:

- a literal dictionary-based gloss
- Google Translate or another public translator, if appropriate for rough comparison
- NLLB-200
- IndicTrans2
- a prompted LLM

Error analysis means reading outputs and categorizing mistakes.

Abshaar error categories

- mistranslation
- missing metaphor
- cultural flattening
- over-interpretation
- hallucinated explanation

- wrong register
- script error
- transliteration error
- grammar issue
- loss of poetic force

Checklist before moving on

- ☐ I can explain what a baseline is.
- ☐ I understand that a fancy model is useless if it is not better than a baseline.
- ☐ I can create error categories for translation outputs.
- ☐ I can manually compare two model outputs.
- ☐ I have created an error-analysis spreadsheet template.

Project checkpoint for Phase 5

By the end of Phase 5, you should be able to explain:

- ☐ What training is.
- ☐ What loss is.
- ☐ What gradient descent is.
- ☐ What learning rate is.
- ☐ What overfitting is.
- ☐ Why Abshaar needs baselines before fine-tuning.

Phase 6. PyTorch and deep learning foundations

Recommended time: 5 to 7 weeks.

PyTorch is the practical bridge between ML theory and real model training. Even if Hugging Face hides much of the training loop, you should understand what is happening underneath.

Lecture 6.1 - Tensors, GPUs, and autograd

What to learn

- tensors
- shapes
- CPU versus GPU
- automatic differentiation
- `.backward()`
- optimizers

Resources

- PyTorch 60 Minute Blitz: https://docs.pytorch.org/tutorials/beginner/deep_learning_60min_blitz.html
- PyTorch tutorials: <https://docs.pytorch.org/tutorials/>
- fast.ai Practical Deep Learning for Coders: <https://course.fast.ai/>

Why Abshaar needs this

Fine-tuning is not magic. A training script loads tensors, computes a loss, calculates gradients, and updates weights or adapter parameters.

Checklist before moving on

- ☐ I know what a tensor is.
- ☐ I understand that models operate on numbers, not raw words.
- ☐ I understand that GPUs speed up matrix operations.
- ☐ I know what autograd does.
- ☐ I can run a PyTorch tutorial notebook.

Lecture 6.2 - Neural network training loop

Conceptual loop

```
for batch in dataloader:
    outputs = model(batch["input_ids"])
    loss = loss_fn(outputs, batch["labels"])
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
```

What each line means

Line	Meaning
<code>outputs = model(...)</code>	model makes predictions
<code>loss = loss_fn(...)</code>	code measures how wrong predictions are
<code>loss.backward()</code>	PyTorch computes gradients
<code>optimizer.step()</code>	parameters are updated
<code>optimizer.zero_grad()</code>	gradients are reset before next batch

Checklist before moving on

- ☐ I can explain a forward pass.
- ☐ I can explain a backward pass.
- ☐ I can explain what an optimizer does.
- ☐ I can explain why gradients must be zeroed.
- ☐ I can train a tiny neural network from a tutorial.

Lecture 6.3 - Checkpoints and reproducibility

What to learn

- saving model checkpoints
- loading checkpoints
- random seeds
- logging loss
- tracking experiments

Why Abshaar needs this

If one fine-tuning run works and another fails, you need to know what changed. Serious ML work requires experiment logs.

Checklist before moving on

- ☐ I can save and load a model checkpoint in a tutorial.
- ☐ I can record learning rate, batch size, and number of epochs.
- ☐ I understand why experiment notes matter.
- ☐ I have a file or spreadsheet for experiment logs.
- ☐ I can explain why reproducibility matters for public research.

Project checkpoint for Phase 6

By the end of Phase 6, you should have:

- ☐ Completed at least one PyTorch beginner tutorial.
- ☐ Trained a tiny model on toy data.
- ☐ Changed the learning rate and observed the effect.
- ☐ Saved and reloaded a model or checkpoint.
- ☐ Written a one-page explanation of the training loop in your own words.

Phase 7. NLP and transformers

Recommended time: 6 to 8 weeks.

Now you begin the technical area closest to Abshaar: natural language processing.

Lecture 7.1 - Tokenization

What to learn

A language model does not directly see words. It sees tokens.

Examples:

```
English word: translation
Possible tokens: translation

English word: unbelievable
Possible tokens: un, believable
```

For Urdu, Punjabi, and Saraiki, tokenization is harder because of:

- script variation
- spelling variation
- optional diacritics
- compound expressions
- code-switching
- borrowed Arabic and Persian vocabulary
- lack of standardized digital corpora

Resources

- Hugging Face course: <https://github.com/huggingface/course>
- Hugging Face LLM course: <https://huggingface.co/learn/llm-course/chapter1/1>
- Hugging Face Tokenizers docs: <https://huggingface.co/docs/tokenizers/index>

Checklist before moving on

- [] I understand that models process tokens, not human words.
- [] I can load a tokenizer from Hugging Face.
- [] I can tokenize an English sentence.
- [] I can tokenize an Urdu or Punjabi line and inspect the tokens.
- [] I can explain why tokenization may be harder for low-resource languages.

Lecture 7.2 - Embeddings

What to learn

Embeddings are vectors that represent meaning or usage patterns. They are essential for search and retrieval.

For example, these questions should retrieve similar notes:

What does Ranjha symbolize?
Why does the poet say he became Ranjha?
Who is Ranjha in this poem?

Resources

- Sentence Transformers: <https://www.sbert.net/>
- BGE-M3 model card: <https://huggingface.co/BAAI/bge-m3>
- BGE-M3 documentation: https://bge-model.com/bge/bge_m3.html

Checklist before moving on

- ☐ I know what an embedding is.
- ☐ I know why embeddings help search by meaning.
- ☐ I can explain cosine similarity in simple terms.
- ☐ I can embed a small list of glossary entries.
- ☐ I can retrieve the most similar glossary entry for a user question.

Lecture 7.3 - Transformers and attention

What to learn

Transformers use attention to decide which tokens are relevant to other tokens. This architecture powers modern translation models and LLMs.

Key terms:

- attention
- self-attention
- encoder
- decoder
- encoder-decoder model
- causal language model
- context window
- sequence length

Resources

- Stanford CS224N: <https://web.stanford.edu/class/cs224n/>
- Hugging Face Transformers docs: <https://huggingface.co/docs/transformers/index>
- The Illustrated Transformer: <https://jalammar.github.io/illustrated-transformer/>

Checklist before moving on

- ☐ I can explain attention without equations.
- ☐ I know the difference between encoder-decoder models and causal LLMs.
- ☐ I know why translation often uses sequence-to-sequence models.
- ☐ I know why instruction-following explanation often uses LLMs.
- ☐ I can load a small transformer model for inference.

Lecture 7.4 - Translation models versus explanation models

What to learn

Abshaar needs at least two model types:

Model type	Main job	Example use
Translation model	Convert source text to target text	Punjabi to English
Instruction LLM	Explain, structure, summarize, answer questions	Tashreeh and metaphor explanation
Embedding model	Retrieve relevant notes	Search glossary and annotations

Checklist before moving on

- ☐ I can explain why one model is not enough.
- ☐ I know which model type is best for raw translation.
- ☐ I know which model type is best for explanations.
- ☐ I know which model type is best for retrieval.
- ☐ I can draw the model architecture for Abshaar on paper.

Project checkpoint for Phase 7

By the end of Phase 7, you should have:

- ☐ Completed the early Hugging Face course chapters.
- ☐ Loaded at least one tokenizer.
- ☐ Tokenized Urdu or Punjabi text.

- [] Embedded at least 10 glossary entries.
- [] Written a short note explaining which model type Abshaar needs for each task.

Phase 8. Translation baselines

Recommended time: 5 to 7 weeks.

This phase produces your first meaningful ML output: baseline translations of real Abshaar material.

Lecture 8.1 - Candidate translation models

Models to test

Model	Why test it	Links
IndicTrans2	Strong Indic translation baseline, especially for supported scheduled languages	https://github.com/AI4Bharat/IndicTrans2 and https://arxiv.org/abs/2305.16307
NLLB-200	Broad multilingual and low-resource translation baseline	https://arxiv.org/abs/2207.04672
OPUS-MT	Useful bilingual baselines where available	https://opus.nlpl.eu/
LLM API or local LLM	Useful as a strong prototype comparator	depends on chosen model

Important caution

Support for a language name does not guarantee good poetry translation. Urdu, Punjabi Shahmukhi, Saraiki, and regional poetic registers need direct testing.

Checklist before moving on

- ☐ I can explain what a translation baseline is.
- ☐ I know why IndicTrans2 and NLLB should be tested.
- ☐ I know that model support does not equal literary quality.
- ☐ I have selected 20 to 50 lines for baseline testing.
- ☐ I have recorded source and license status for those lines.

Lecture 8.2 - Running a baseline experiment

Experiment design

For each line, save:

```
line_id
source_language
script
original_text
human_reference_if_available
model_name
model_version
prompt_or_settings
model_output
date_run
notes
```

Output table example

line_id	model_name	output	initial notes
bulleh_001_l001	NLLB-200	...	misses repetition
bulleh_001_l001	IndicTrans2	...	better literal sense
bulleh_001_l001	LLM	...	stronger explanation but over-interprets

Checklist before moving on

- ☐ I can run at least one translation model or API on a poem line.
- ☐ I save outputs systematically.
- ☐ I record model name and settings.
- ☐ I do not overwrite previous outputs.
- ☐ I can compare outputs for the same line side by side.

Lecture 8.3 - Translation output types

What to generate

For Abshaar, do not ask for only one translation. Generate several layers:

Output layer	Purpose
word gloss	helps learners see the source structure
literal translation	preserves direct meaning
reader translation	natural English
literary translation	preserves poetic force
uncertainty note	marks ambiguity

Example prompt for an LLM baseline

You are assisting with a South Asian poetry translation project.
 Translate the following line carefully.
 Separate literal translation from interpretation.
 Do not claim mystical meaning as literal meaning.

Original language: Punjabi

Script: Shahmukhi

Line: ਰਾਜਹਾ ਰਾਜਹਾ ਕਰਦੀ ਨੀ ਮੇਰੇ ਰਾਜਹਾ ਹੋਈ

Return:

1. Roman transliteration
2. Literal translation
3. Reader-friendly translation
4. Literary translation
5. Notes on ambiguity

Checklist before moving on

- ☐ I can produce multiple translation layers.
- ☐ I understand why a literary translation is not the same as a literal translation.
- ☐ I know how over-interpretation can enter translation.
- ☐ I have a prompt template for baseline LLM translation.
- ☐ I have tested the prompt on at least 10 lines.

Project checkpoint for Phase 8

By the end of Phase 8, you should have:

- ☐ 20 to 50 test lines.
- ☐ Outputs from at least two baseline systems.
- ☐ A spreadsheet comparing outputs.
- ☐ Notes on common error types.
- ☐ A first recommendation on which baseline is strongest for your data.

Phase 9. Evaluation and human review

Recommended time: 5 to 7 weeks.

Evaluation is how you avoid fooling yourself. Poetry translation cannot be measured only by automatic scores.

Lecture 9.1 - Automatic metrics

What to learn

Automatic metrics can help but cannot judge the full value of poetry translation.

Useful tools:

- SacreBLEU: <https://github.com/mjpost/sacrebleu>
- COMET: <https://unbabel.github.io/COMET/html/index.html>
- BLEURT: <https://github.com/google-research/bleurt>

What they are good for

- comparing systems on literal translation
- detecting large regressions
- checking whether a fine-tune made outputs worse

What they are weak at

- metaphor
- spiritual ambiguity
- poetic music
- register
- cultural context
- multiple valid translations

Checklist before moving on

- ☐ I understand that BLEU is not enough for poetry.
- ☐ I can explain why automatic metrics are still useful.
- ☐ I know what a reference translation is.
- ☐ I can run or at least interpret a basic automatic score.
- ☐ I can explain why human review is required.

Lecture 9.2 - Human evaluation rubric

Recommended 1 to 5 scoring dimensions

Dimension	Question
literal accuracy	Does it preserve the basic meaning?
poetic force	Does the English carry emotional energy?
cultural fidelity	Does it avoid flattening references?
metaphor preservation	Does it preserve core images?
register	Does it preserve mystical, folk, devotional, or ironic tone?
readability	Can an English reader understand it?
honesty	Does it separate translation from interpretation?

Checklist before moving on

- ☐ I have created a scoring rubric.
- ☐ I can score one translation using the rubric.
- ☐ I can explain the difference between accuracy and poetic force.
- ☐ I can explain why honesty is a separate category.
- ☐ I have evaluated at least 20 model outputs manually.

Lecture 9.3 - MQM-style error annotation

What to learn

MQM, or Multidimensional Quality Metrics, gives a structured way to classify translation errors. You can adapt it for Abshaar.

Resource:

- MQM: <https://themqm.org/>

Abshaar error categories

```
accuracy error
fluency error
terminology error
metaphor loss
cultural flattening
over-interpretation
under-interpretation
hallucinated context
register mismatch
script error
transliteration error
```

Error example

```
{
  "line_id": "bulleh_001_l001",
  "model_output": "I became God.",
  "error_type": "over-interpretation",
  "severity": "major",
  "comment": "The original says the speaker became Ranjha. A mystical reading is possible,
but should not replace the literal translation."
}
```

Checklist before moving on

- ☐ I can classify errors by type.
- ☐ I can mark severity.
- ☐ I can distinguish a wrong translation from an overconfident interpretation.
- ☐ I have created an error taxonomy for Abshaar.
- ☐ I have annotated at least 20 errors.

Lecture 9.4 - Human review workflow

Workflow

1. Model produces draft.
2. Reviewer edits translation and notes.
3. Reviewer marks error types.
4. Corrected version becomes reviewed data.
5. Reviewed data can later be used for fine-tuning.

Review record example

```
{
  "line_id": "bulleh_001_l001",
  "model_name": "baseline_llm_v1",
  "model_translation": "...",
  "corrected_literal_translation": "...",
  "corrected_literary_translation": "...",
  "reviewer": "reviewer_001",
  "review_status": "human_reviewed",
  "error_tags": ["over_interpretation", "register_mismatch"],
  "review_notes": "Keep Ranjha literal before offering Sufi interpretation."
}
```

Checklist before moving on

- ☐ I have a review form or spreadsheet.
- ☐ I store model output and human correction separately.
- ☐ I record reviewer notes.
- ☐ I mark review status clearly.
- ☐ I have at least 50 reviewed examples before thinking seriously about fine-tuning.

Project checkpoint for Phase 9

By the end of Phase 9, you should have:

- ☐ A translation evaluation rubric.
- ☐ An error taxonomy.
- ☐ 50 to 100 manually reviewed outputs.
- ☐ A gold dataset folder separate from drafts.
- ☐ A short report comparing baseline model performance.

Phase 10. RAG for source-grounded explanations

Recommended time: 6 to 8 weeks.

RAG stands for retrieval-augmented generation. It means the model answers after searching your approved notes and sources.

Lecture 10.1 - Why RAG before fine-tuning

What to learn

A model should not be trusted to invent cultural context. RAG gives it a library to consult.

Instead of asking:

What does Ranjha mean?

and letting the model guess, the system should retrieve:

- glossary note on Ranjha
- note on Heer-Ranjha
- poem-specific annotation
- source note on Sufi beloved symbolism

Then the model answers using those notes.

Resource

- Original RAG paper: <https://arxiv.org/abs/2005.11401>
- LlamaIndex RAG guide: <https://developers.llamaindex.ai/python/framework/understanding/rag/>

Checklist before moving on

- [] I can explain RAG in plain English.
- [] I know why RAG reduces hallucination.
- [] I know why source-grounded explanation matters for Abshaar.
- [] I can list the documents that should go into the retrieval library.
- [] I understand that RAG does not replace human review.

Lecture 10.2 - Glossary and context notes

What to build

Create notes like this:

```
{
  "id": "glossary_ranjha",
  "term": "Ranjha",
  "category": "romance_reference",
  "literal_meaning": "Male beloved from the Heer-Ranjha romance.",
  "interpretive_meaning": "In Punjabi Sufi poetry, Ranjha can function as beloved, self, or sign of mystical absorption depending on context.",
  "confidence": "medium",
  "source_ids": ["source_001"],
  "review_status": "needs_review"
}
```

First glossary terms

Start with:

- Ranjha
- Heer
- ishq
- ishq-e-majazi
- ishq-e-haqiqi
- murshid
- faqir
- mullah
- Kaaba
- masjid
- hijr
- visal
- nafs
- fana
- sama

Checklist before moving on

- ☐ I have created at least 25 glossary entries.
- ☐ Each glossary entry has source IDs.
- ☐ Each entry separates literal meaning from interpretation.
- ☐ Each entry has review status.
- ☐ I can explain why ambiguous terms need confidence labels.

Lecture 10.3 - Embedding and vector databases

Tools to consider

Tool	Use case	Link
Chroma	easiest local vector database for a prototype	https://docs.trychroma.com/docs/overview/introduction
Qdrant	stronger production vector database	https://qdrant.tech/documentation/
Supabase Vector	good if you want Postgres plus embeddings	https://supabase.com/modules/vector
BGE-M3	multilingual embedding model to test	https://huggingface.co/BAAI/bge-m3

RAG pipeline

1. Create notes.
2. Split notes into chunks.
3. Embed chunks.
4. Store embeddings in a vector database.
5. Embed user question.
6. Retrieve top-k relevant notes.
7. Ask LLM to answer using retrieved notes only.
8. Show citations or source IDs.

Checklist before moving on

- ☐ I can embed a glossary note.
- ☐ I can store embeddings in a local vector database.
- ☐ I can retrieve top-k notes for a question.
- ☐ I can pass retrieved notes into a prompt.
- ☐ I can show which notes supported the answer.

Lecture 10.4 - RAG prompt design

Prompt template

You are Abshaar, a careful assistant for South Asian poetry.
Use only the retrieved notes below.
Separate literal meaning from interpretation.
If the notes do not support an answer, say that the answer is uncertain.
Do not invent sources.

Question:
{user_question}

Retrieved notes:
{retrieved_notes}

Answer format:
1. Short answer
2. Literal meaning
3. Possible interpretation
4. What is uncertain
5. Sources used

Checklist before moving on

- ☐ My prompt tells the model to use retrieved notes.
- ☐ My prompt tells the model not to invent sources.
- ☐ My prompt separates fact from interpretation.
- ☐ My prompt includes uncertainty.
- ☐ I have tested at least 10 user questions.

Project checkpoint for Phase 10

By the end of Phase 10, you should have:

- ☐ 50 to 100 glossary or context notes.
- ☐ A local vector index.
- ☐ A script that retrieves relevant notes.
- ☐ A RAG answer script.
- ☐ A small test set of user questions and expected supporting notes.

Phase 11. Fine-tuning

Recommended time: 8 to 12 weeks after you have reviewed data.

Fine-tuning should come after baselines, review, and RAG. Do not start here.

Lecture 11.1 - Types of training

Pretraining

Training a model from scratch on huge amounts of text. This is not your starting point.

Fine-tuning

Taking a pretrained model and training it further on your data.

Supervised fine-tuning

Training on input-output examples.

LoRA and QLoRA

LoRA trains small adapter weights instead of updating the whole model. QLoRA uses quantization to reduce memory use.

Resources

- Hugging Face PEFT LoRA guide: https://huggingface.co/docs/peft/developer_guides/lora
- LoRA paper: <https://arxiv.org/abs/2106.09685>
- QLoRA paper: <https://arxiv.org/abs/2305.14314>
- Hugging Face TRL SFTTrainer: https://huggingface.co/docs/trl/sft_trainer
- bitsandbytes quantization docs: <https://huggingface.co/docs/transformers/quantization/bitsandbytes>

Checklist before moving on

- ☐ I can explain why pretraining from scratch is unrealistic for the first version.
- ☐ I can explain fine-tuning.
- ☐ I can explain supervised fine-tuning.
- ☐ I can explain LoRA in plain English.
- ☐ I know why QLoRA helps with limited GPU memory.

Lecture 11.2 - Fine-tuning target options

Option A: translation model fine-tune

Input:

Punjabi, Saraiki, or Urdu original line

Output:

English translation

Good for literal translation. Less good for rich explanation.

Option B: instruction LLM fine-tune

Input:

Translate and explain this line using Abshaar format.

Output:

Literal translation
Literary translation
Tashreeh
Glossary notes
Uncertainty
Sources

Better for the full Abshaar product.

Recommendation

Start with instruction-style fine-tuning only after you have enough reviewed examples. Use RAG for source grounding even after fine-tuning.

Checklist before moving on

- [] I know the difference between translation fine-tuning and instruction fine-tuning.
- [] I know which one better matches Abshaar's full output.
- [] I know that fine-tuning does not remove the need for RAG.
- [] I have chosen one fine-tuning target for the first experiment.
- [] I have written down why I chose it.

Lecture 11.3 - Training data format

Example instruction dataset row

```
{
  "messages": [
    {
      "role": "system",
      "content": "You are Abshaar, a careful South Asian poetry translation assistant. Separate literal translation from interpretation. Cite sources when available."
    },
    {
      "role": "user",
      "content": "Translate and explain this Punjabi Sufi line: رانجھا رانجھا کردی نی میں آہے رانجھا ہوئی"
    },
    {
      "role": "assistant",
      "content": "Literal translation: Saying Ranjha, Ranjha, I myself became Ranjha. \nLiterary translation: Calling Ranjha's name again and again, I became Ranjha myself. \nTashreeh: The line uses repeated naming to suggest absorption in the beloved. Ranjha should first be understood as the beloved from Heer-Ranjha; a Sufi reading is possible but should be marked as interpretation. \nUncertainty: The strength of the mystical reading depends on poem context."
    }
  ]
}
```

Dataset size rules of thumb

Reviewed examples	What they are useful for
50 to 100	toy experiment only
500	early style test
2,000	meaningful first adaptation
10,000	serious adaptation

Quality matters more than quantity. Five hundred excellent examples may beat thousands of messy examples.

Checklist before moving on

- ☐ I can create JSONL training examples.
- ☐ I separate train, validation, and test files.
- ☐ I only train on reviewed examples.
- ☐ I keep auto-generated drafts out of gold training data.
- ☐ I have at least a small validation set before training.

Lecture 11.4 - Hyperparameters

Key hyperparameters

Hyperparameter	What it controls	Beginner intuition
learning_rate	size of each update	too high can break learning, too low learns slowly
num_train_epochs	passes over training data	too many can overfit
batch_size	examples per device step	bigger uses more memory
gradient_accumulation_steps	simulates larger batches	useful on small GPUs
max_seq_length	maximum tokens per example	long notes need longer sequences
warmup_ratio	gradual learning-rate start	can stabilize training
weight_decay	regularization	can reduce overfitting
lora_r	adapter rank	higher means more capacity and memory
lora_alpha	adapter scaling	controls adapter strength
lora_dropout	dropout for adapters	can help generalization
eval_steps	validation frequency	catches overfitting
save_steps	checkpoint frequency	protects work

Checklist before moving on

- ☐ I can define learning rate.
- ☐ I can define epoch.
- ☐ I can define batch size.
- ☐ I can define max sequence length.
- ☐ I can explain why validation loss matters during fine-tuning.

Lecture 11.5 - Simplified LoRA training script

This example is conceptual. Model names, target modules, and memory settings must be checked for your actual environment.

```

from datasets import load_dataset
from transformers import AutoModelForCausalLM, AutoTokenizer, TrainingArguments
from peft import LoraConfig, get_peft_model
from trl import SFTTrainer

model_name = "Qwen/Qwen3-0.6B"

dataset = load_dataset("json", data_files={
    "train": "data/train.jsonl",
    "validation": "data/validation.jsonl"
})

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name, device_map="auto")

lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    lora_dropout=0.05,
    target_modules=["q_proj", "v_proj"],
    task_type="CAUSAL_LM"
)

model = get_peft_model(model, lora_config)

args = TrainingArguments(
    output_dir="outputs/abshaar-lora",
    learning_rate=2e-4,
    num_train_epochs=3,
    per_device_train_batch_size=1,
    gradient_accumulation_steps=16,
    eval_strategy="steps",
    eval_steps=100,
    save_steps=100,
    logging_steps=10,
    report_to="none"
)

trainer = SFTTrainer(
    model=model,
    args=args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["validation"]
)

trainer.train()

```

Checklist before moving on

- ☐ I can identify where the model is loaded.
- ☐ I can identify where the dataset is loaded.
- ☐ I can identify the LoRA configuration.
- ☐ I can identify the training arguments.
- ☐ I understand that this script should only be run after preparing reviewed data.

Project checkpoint for Phase 11

By the end of Phase 11, you should have:

- ☐ A reviewed training set.
- ☐ A validation set.
- ☐ A test set.
- ☐ A first small LoRA experiment.
- ☐ Before-and-after examples comparing base model and fine-tuned model.
- ☐ A report of failures and regressions.

Phase 12. Website MVP

Recommended time: 8 to 12 weeks.

The website is where Abshaar becomes public. Do not wait for perfect fine-tuning before building a small public demo. The first MVP can use reviewed translations plus RAG.

Lecture 12.1 - Fast demo route

Recommended stack

- Gradio for interface.
- Hugging Face Spaces for hosting.
- Chroma for local retrieval.
- A model API or local model for answers.

Resources:

- Gradio docs: <https://www.gradio.app/docs>
- Hugging Face Spaces with Gradio: <https://www.gradio.app/guides/using-hugging-face-integrations>

What the demo should do

- select poet
- select poem
- show original text
- show transliteration
- show literal translation
- show literary translation
- show notes
- answer questions using RAG

Checklist before moving on

- ☐ I can run a small Gradio app locally.
- ☐ I can display one poem in the app.
- ☐ I can add a question box.
- ☐ I can connect the question box to a simple RAG answer.
- ☐ I can explain the limitations of the demo.

Lecture 12.2 - Serious web app route

Recommended stack

Layer	Tool
frontend	Next.js
backend	FastAPI
database	PostgreSQL or Supabase
vector search	Supabase Vector, Qdrant, or Chroma
model serving	API first, local model later
deployment	Vercel plus backend host, or a unified platform

Resources:

- Next.js docs: <https://nextjs.org/docs>
- FastAPI docs: <https://fastapi.tiangolo.com/>
- Supabase docs: <https://supabase.com/docs>

App architecture

```
Browser
  -> Next.js frontend
    -> FastAPI backend
      -> poem database
      -> vector database
      -> translation model or stored translations
      -> explanation model
```

Checklist before moving on

- ☐ I know what frontend means.
- ☐ I know what backend means.
- ☐ I know what a database does.
- ☐ I know why model calls should usually happen on the backend.
- ☐ I can explain how a user question travels through the system.

Lecture 12.3 - Poem page design

Recommended page layout

```
Poem title and poet
Source and license note
Original script
Roman transliteration
Literal translation
Reader translation
Literary translation
Tashreeh
Glossary terms
Related concepts
Question box
Retrieved sources used in answer
Correction or feedback button
```

Checklist before moving on

- ☐ Every poem page shows source information.
- ☐ Translation layers are visually separated.
- ☐ Glossary terms are easy to find.
- ☐ The Q&A answer shows sources or note IDs.
- ☐ Users can report a translation or explanation issue.

Lecture 12.4 - Review dashboard

Why this matters

A public project needs a correction workflow. Users and scholars should be able to suggest changes, but changes should not go live without review.

Minimum review fields

```
item_id
item_type
current_text
suggested_text
reason
submitter
review_status
reviewer_notes
created_date
```

Checklist before moving on

- ☐ I have a way to collect corrections.

- ☐ Suggestions do not automatically become published content.
- ☐ I can mark suggestions as pending, accepted, or rejected.
- ☐ I can turn accepted corrections into reviewed dataset examples.
- ☐ I understand that the review loop improves both the website and future models.

Project checkpoint for Phase 12

By the end of Phase 12, you should have:

- ☐ A public or local demo.
- ☐ 10 polished poems.
- ☐ Original text, transliteration, translations, and notes.
- ☐ RAG question answering.
- ☐ A feedback or review workflow.
- ☐ A clear limitations page.

Phase 13. Launch, governance, and long-term growth

Recommended time: ongoing.

A public cultural project must be careful, transparent, and maintainable.

Lecture 13.1 - Trust and transparency

What to include publicly

- data sources
- license status
- review status
- model used
- limitations
- how to report errors
- how to contribute
- how translations were produced

Checklist before moving on

- ☐ Every public poem has source information.
- ☐ Users can distinguish human-reviewed content from model drafts.
- ☐ The site explains that interpretations may be multiple.
- ☐ The site does not present uncertain readings as facts.
- ☐ There is a public correction process.

Lecture 13.2 - Community contribution

Possible contributor roles

Role	Contribution
native speaker	checks meaning and idiom
literature student	adds context notes
scholar	reviews interpretations
developer	improves data and website
designer	improves user experience
community reader	reports errors

Checklist before moving on

- ☐ I have written contribution guidelines.
- ☐ I have a reviewer role system.
- ☐ I have a way to credit contributors.
- ☐ I have a moderation policy.
- ☐ I have a plan for resolving disagreements about interpretation.

Lecture 13.3 - Research and publication potential

Possible outputs

- public website
- open dataset
- paper on low-resource literary translation
- paper on source-grounded cultural explanation
- benchmark for South Asian poetry translation
- educational tool for diaspora readers

Checklist before moving on

- ☐ I keep experiment logs.
- ☐ I keep dataset version numbers.
- ☐ I keep model comparison reports.
- ☐ I can reproduce key experiments.

- ☐ I have a roadmap for turning the project into research if desired.

Appendix A. One-year calendar

Month	Focus	Deliverable
1	Setup, Git, Python basics	working repo and first scripts
2	Python files, JSON, CSV, Pandas	poem JSON and review spreadsheet
3	Corpus design	10 structured poems and source registry
4	Data validation	validation scripts and quality labels
5	ML foundations	written explanation of loss and training
6	PyTorch	toy neural network and training loop understanding
7	NLP and transformers	tokenization and embedding experiments
8	Translation baselines	NLLB, IndicTrans2, and LLM comparison
9	Evaluation	rubric, error taxonomy, 50 reviewed examples
10	RAG	glossary index and question-answering prototype
11	Fine-tuning preparation	train, validation, and test JSONL files
12	MVP	Bulleh Shah Explorer with 10 to 20 polished works

Appendix B. First 10 concrete Abshaar projects

1. Poem JSON builder

Create one clean poem JSON file.

Checklist:

- ☐ Has ID.
- ☐ Has title.
- ☐ Has poet.
- ☐ Has language.
- ☐ Has script.
- ☐ Has original text.
- ☐ Has source ID.
- ☐ Has review status.

2. Source registry

Create `source_registry.json`.

Checklist:

- ☐ Every source has URL.
- ☐ Every source has access date.
- ☐ Every source has license status.
- ☐ Every source has allowed-use notes.

3. Line splitter

Write a script that splits a poem into line entries.

Checklist:

- ☐ Each line has line ID.
- ☐ Each line links back to poem ID.
- ☐ Empty lines are handled.
- ☐ Output is saved as JSONL.

4. Translation review spreadsheet

Create a spreadsheet for model outputs and human corrections.

Checklist:

- ☐ Has line ID.
- ☐ Has model name.
- ☐ Has model output.
- ☐ Has human correction.
- ☐ Has error tags.

5. Glossary v0.1

Create 25 glossary entries.

Checklist:

- ☐ Literal meaning.
- ☐ Interpretive meaning.
- ☐ Source ID.
- ☐ Confidence.
- ☐ Review status.

6. Baseline translation experiment

Run two models or one model plus one LLM prompt on the same 20 lines.

Checklist:

- ☐ Same input lines.
- ☐ Outputs saved.
- ☐ Model names recorded.
- ☐ Manual notes added.

7. Evaluation rubric

Create a 1 to 5 scoring rubric.

Checklist:

- ☐ Literal accuracy.
- ☐ Poetic force.
- ☐ Cultural fidelity.
- ☐ Register.

- ☐ Honesty.

8. Local RAG prototype

Embed glossary notes and retrieve relevant notes.

Checklist:

- ☐ Notes embedded.
- ☐ User question embedded.
- ☐ Top-k notes retrieved.
- ☐ Answer generated with retrieved notes.
- ☐ Sources shown.

9. Static poem page

Build one HTML or Markdown page for one poem.

Checklist:

- ☐ Original text.
- ☐ Transliteration.
- ☐ Literal translation.
- ☐ Literary translation.
- ☐ Notes.
- ☐ Sources.

10. First fine-tuning dataset

Create a tiny instruction-style JSONL dataset from reviewed examples.

Checklist:

- ☐ Only reviewed examples.
- ☐ System message included.
- ☐ User message included.
- ☐ Assistant answer included.
- ☐ Train and validation files separated.

Appendix C. Model decision matrix

Task	First choice	Backup	Notes
Urdu/Punjabi/Saraiki to English baseline	IndicTrans2 where language support fits	NLLB-200	Test on your exact scripts and registers
Broad multilingual fallback	NLLB-200	OPUS-MT models	Useful when IndicTrans2 does not fit
Explanation and <i>tashreeh</i>	instruction LLM	smaller local LLM	Must use RAG for source grounding
Retrieval	BGE-M3	multilingual E5 or Sentence Transformers	Test retrieval quality on Urdu/Punjabi queries
Fine-tuning	LoRA or QLoRA	prompt-only plus RAG	Fine-tune only after reviewed data
Public demo	Gradio	simple Next.js page	Start simple
Production app	Next.js plus FastAPI	full-stack framework	Add database and review workflow

Appendix D. Minimum technical vocabulary

Term	Meaning
token	unit of text processed by model
tokenizer	tool that converts text to tokens
embedding	vector representation of text
vector database	database for similarity search over embeddings
corpus	structured collection of texts
dataset	examples used for training or evaluation
label	correct answer or target output
loss	measure of model error
gradient	direction for reducing loss
learning rate	size of training update
epoch	one pass through training data
batch	group of examples processed together
overfitting	model memorizes training examples and fails on new data
baseline	comparison system before improvement
RAG	retrieval-augmented generation
SFT	supervised fine-tuning
LoRA	adapter-based fine-tuning method
QLoRA	quantized LoRA fine-tuning method
checkpoint	saved model state
inference	using a trained model to produce output

Appendix E. Resource library

Beginner programming

- CS50 Python: <https://cs50.harvard.edu/python/>
- CS50x: <https://cs50.harvard.edu/x/>
- Kaggle Python: <https://www.kaggle.com/learn/python>
- Kaggle Pandas: <https://www.kaggle.com/learn/pandas>
- Python tutorial: <https://docs.python.org/3/tutorial/>

Git, GitHub, and development tools

- GitHub Skills: <https://skills.github.com/>
- Pro Git book: <https://git-scm.com/book/en/v2>
- VS Code docs: <https://code.visualstudio.com/docs>
- Abshaar repo: <https://github.com/RaufNawaz/Abshaar>

ML foundations

- Google ML Crash Course: <https://developers.google.com/machine-learning/crash-course>
- Linear regression: <https://developers.google.com/machine-learning/crash-course/linear-regression>
- Gradient descent: <https://developers.google.com/machine-learning/crash-course/linear-regression/gradient-descent>
- Kaggle Intro to Machine Learning: <https://www.kaggle.com/learn/intro-to-machine-learning>

Deep learning

- PyTorch 60 Minute Blitz: https://docs.pytorch.org/tutorials/beginner/deep_learning_60min_blitz.html
- PyTorch tutorials: <https://docs.pytorch.org/tutorials/>
- fast.ai: <https://course.fast.ai/>

NLP and transformers

- Hugging Face course: <https://github.com/huggingface/course>
- Hugging Face LLM course: <https://huggingface.co/learn/llm-course/chapter1/1>
- Hugging Face Transformers: <https://huggingface.co/docs/transformers/index>

- Hugging Face Datasets: <https://huggingface.co/docs/datasets/index>
- Stanford CS224N: <https://web.stanford.edu/class/cs224n/>
- The Illustrated Transformer: <https://jalammar.github.io/illustrated-transformer/>

Translation models and corpora

- IndicTrans2 GitHub: <https://github.com/AI4Bharat/IndicTrans2>
- IndicTrans2 paper: <https://arxiv.org/abs/2305.16307>
- NLLB paper: <https://arxiv.org/abs/2207.04672>
- OPUS: <https://opus.nlpl.eu/>
- FLORES-200: <https://github.com/facebookresearch/flores/blob/main/flores200/README.md>
- UMC005 English-Urdu: <https://ufal.mff.cuni.cz/umc/005-en-ur/>

RAG and retrieval

- RAG paper: <https://arxiv.org/abs/2005.11401>
- BGE-M3 model card: <https://huggingface.co/BAAI/bge-m3>
- BGE-M3 docs: https://bge-model.com/bge/bge_m3.html
- Sentence Transformers: <https://www.sbert.net/>
- Chroma docs: <https://docs.trychroma.com/docs/overview/introduction>
- Qdrant docs: <https://qdrant.tech/documentation/>
- Supabase Vector: <https://supabase.com/modules/vector>
- LlamaIndex RAG guide: <https://developers.llamaindex.ai/python/framework/understanding/rag/>

Fine-tuning

- PEFT LoRA guide: https://huggingface.co/docs/peft/developer_guides/lora
- LoRA paper: <https://arxiv.org/abs/2106.09685>
- QLoRA paper: <https://arxiv.org/abs/2305.14314>
- TRL SFTTrainer: https://huggingface.co/docs/trl/sft_trainer
- bitsandbytes quantization: <https://huggingface.co/docs/transformers/quantization/bitsandbytes>

Evaluation

- SacreBLEU: <https://github.com/mjpost/sacrebleu>
- COMET: <https://unbabel.github.io/COMET/html/index.html>
- BLEURT: <https://github.com/google-research/bleurt>
- MQM: <https://themqm.org/>

Website and deployment

- Gradio: <https://www.gradio.app/docs>
- Gradio with Hugging Face Spaces: <https://www.gradio.app/guides/using-hugging-face-integrations>
- FastAPI: <https://fastapi.tiangolo.com/>
- Next.js: <https://nextjs.org/docs>
- Supabase: <https://supabase.com/docs>

Appendix F. Final readiness checklist

You are ready to call the first Abshaar MVP successful when:

- ☐ You have 10 to 20 polished poems.
- ☐ Every poem has source information.
- ☐ Every poem has original text, transliteration, literal translation, literary translation, and notes.
- ☐ Every explanation is separated from translation.
- ☐ Every glossary entry has source or review status.
- ☐ The Q&A system retrieves notes before answering.
- ☐ Users can see limitations.
- ☐ Users can submit corrections.
- ☐ You have an evaluation rubric.
- ☐ You have a path from user corrections to better reviewed data.

The right first public target is still:

Bulleh Shah Explorer: 10 to 20 carefully reviewed works with original text, transliteration, translation layers, glossary, *tashreeh*, and source-grounded Q&A.

After that, expand slowly to more poets, more languages, more scripts, and better models.